

src/app/docsQuality.ts · Code explanation (L1-248)

Kind: modified · Baseline: 8e0dc031dcfc · Target: NovaDiff · Requested mode: exact · Effective mode: exact ·
Generated: 2026-05-19T10:03:38.560Z

What it does `docsQuality.ts` generates contextual prompts and a release-overview markdown for a code-diff report.- `buildSummaryPromptContext` formats up to 10 `SummaryIndexEntry` objects into a concise list for LLM synthesis.- `buildRiskPromptContext` turns up to 14 `RiskSignal` objects into a deterministic risk hint block.- `deriveConfidenceBadges` creates evidence-badge objects based on file-change coverage, risk severity, and selection docs.- ``buildRelease`

Overview

Markdown` stitches together a full markdown document that lists changed files, risk tables, confidence badges, and summary snippets.

How it works1. Utility helpers - `uniqStrings` removes duplicates and limits string arrays. - `compactText` normalises whitespace and truncates to a max length, appending an ellipsis. - `relPathAnchorId` slugifies a relative path for anchor links.2. **Context builders*

- - `buildSummaryPromptContext` slices the first `limit` entries, extracts badge labels, compacts markdown, and prefixes the list with a header. - `buildRiskPromptContext` slices the first `limit` signals, pulls up to 3 evidence strings, and formats each with severity, confidence, and title.3. **Badge derivation*
 - - `deriveConfidenceBadges` calculates `summaryCoverage` as `summaries.length / rows.length`. - It pushes badges for coverage level, presence of high-severity signals, any risk signals, and available selection docs. - A heuristic-scan badge is always added.4. **Markdown assembly*
- - ``buildRelease`

Overview

Markdown` groups risks by severity, limits top files, summaries, and selections, and builds tables and sections using the helper functions above. - The final string is a concatenated array of markdown lines.

Notes for reviewers- The code limits output sizes (`limit` parameters) to keep prompts concise.- Badge keys are unique; `push` guards against duplicates.- All string manipulations use `String(value ?? "")` to avoid `undefined` errors.- The

final markdown includes a hard-coded advisory note about future enrichment.- No external dependencies are used; all logic is self-contained.